

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Balan Gabriel FAF-241

Report

Laboratory Work No. 1

Operating Systems

Topic: The OS as a Resource Manager

Checked by:

Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Initial setup **3**

2 Part 1 — Files and directories **3**
2.1 Observations 4

3 Part 2 — Processes **4**
3.1 Observations 5

4 Part 3 — Memory **6**
4.1 Observations 6

5 Part 4 — Devices and storage **7**
5.1 Observations 7

6 Wrap-up **8**

7 Conclusion **8**

1 Initial setup

```
mkdir -p ~/os-lab1 && cd ~/os-lab1
whoami
uname -a
```

Output:

```
findredhundred
Linux Ubuntu 7.0.0-34-generic #34-Ubuntu SMP PREEMPT_DYNAMIC Wed Sep  2
↪  14:29:37 UTC 2026 x86_64 GNU/Linux
up 17 min, 1 user
```

I created a separate folder for the lab, checked my username, the running kernel and how long the system had been up. `mkdir -p` created the folder if needed, while `&&` made `cd` run only if the first command succeeded.

2 Part 1 — Files and directories

```
pwd
ls -la /
ls -la ~
cd /etc && ls | head
```

Output:

```
/home/findredhundred/os-lab1
```

`pwd` showed my current directory. `ls -la` showed long file information, including hidden files, and `ls | head` only made it show a few lines so i don't have my whole screen clogged with garbage.

```
cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
```

Output:

```
-rw-rw-r-- 1 findredhundred findredhundred 24 Sep 29 01:10 note.txt
```

```
-rw-rw-r-- 1 findredhundred findredhundred 24 Sep 29 01:11 renamed.txt
```

Here I created a directory and a text file, copied it, renamed the copy and then removed it. `echo` produced text and `>` redirected that text into `note.txt`. as `note.txt` wasn't yet present, the os created the file!

```
ls -l note.txt
chmod 600 note.txt
ls -l note.txt
chmod 644 note.txt
```

Output:

```
-rw------- 1 findredhundred findredhundred 0 Sep 29 01:30 note.txt
-rw-r--r-- 1 findredhundred findredhundred 0 Sep 29 01:30 note.txt
```

`chmod` changed the permissions of the file. In `600`, the owner has read and write permissions and everyone else has none. In `644`, the owner can read and write, while group and others can only read.

2.1 Observations

The files I created were owned by `findredhundred` and the group was also `findredhundred`. The first character in `-rw-------` is the file type, while the remaining nine characters are permissions for owner, group and others. Two directories directly under `/` are `/tmp`, which stores temporary files, and `/etc`, which contains system-wide configuration. This part showed how Linux organizes files in one directory tree and controls access using ownership and permissions.

3 Part 2 — Processes

```
ps aux | head
ps aux | wc -l
top
```

Output:

```
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2  26716  8012 ?        Ss   06:51   0:05
↪  /usr/lib/systemd/systemd --switched-root --system --deserialize=51
root           2  0.0  0.0      0     0 ?        S    06:51   0:00
↪  [kthreadd]
root           3  0.0  0.0      0     0 ?        S    06:51   0:00
↪  [pool_workqueue_release]
```

```
root           4  0.0  0.0      0     0 ?        I<   06:51   0:00
↪  [kworker/R-rcu_gp]
```

`ps` showed running processes, `wc -l` counted the lines, and `top` showed a live view of process, CPU and memory activity.

```
sleep 300 &
jobs
ps -ef | grep sleep
kill 14234
```

Output:

```
[1] 35145
[1]+  Running                 sleep 300 &
findred+   35145   18042  0 15:31 pts/1    00:00:00 sleep 300
findred+   35157   18042  0 15:31 pts/1    00:00:00 grep --color=auto
↪  sleep
[1]+  Terminated              sleep 300
```

`sleep 300 &` started a process in the background. `jobs` showed the shell job, `ps -ef | grep sleep` found it in the full process list, and `kill` terminated it using its PID.

```
sleep 300 &
PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20
kill $PID
```

Output:

```
Name:   sleep
State:  S (sleeping)
Pid:    19535
PPid:   18042
```

`$!` contained the PID of the last background process, so I saved it in `PID`. The `/proc/$PID` directory exposed live information about that process, including its state, parent PID, memory data and other kernel-maintained information.

3.1 Observations

PID 1 on my system was `systemd`, which is the main system manager started during boot. `ps aux | wc -l` gave roughly 230 running processes, although the exact number changed between runs. The `sleep` process had the state `S (sleeping)`, which made sense because

it was waiting instead of actively using the CPU. This part showed how the OS keeps track of processes, assigns PIDs and exposes process information through `/proc`.

4 Part 3 — Memory

```
free -h
cat /proc/meminfo | head -6
```

Output:

```
total        used        free      shared  buff/cache   availableMem:
3.3Gi       2.8Gi       110Mi        64Mi       610Mi       524MiSwap:
   0B          0B          0B
```

```
MemTotal:        3479668 kB
MemFree:          107116 kB
MemAvailable:     518232 kB
Buffers:            6836 kB
Cached:           551916 kB
SwapCached:            0 kB
```

`free -h` gave a readable summary of RAM and swap. `/proc/meminfo` showed the same information in more detail. I noticed that `free` memory was much lower than `available` memory because Linux was using some RAM for cache that could be reused when needed.

```
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
kill $PID
```

Output:

```
VmRSS: 8016 kB
```

`VmRSS` showed how much physical RAM was resident for the process. Even a simple sleeping process still used several megabytes for its executable, libraries, stack and other mappings.

4.1 Observations

The VM had about 3.3 GiB of RAM. Around 100 MiB was completely free at one point, but much more was still available because cached memory can be reassigned. No swap was configured, so the swap size was `0B`. The `sleep` process used about 8 MB of resident memory, which was more than I expected for a process that was mostly just waiting. This

part helped me see the difference between total, free, available and process memory.

5 Part 4 — Devices and storage

```
df -h
lsblk
du -sh ~/os-lab1
ls -l /dev | head
mount | head
```

Output:

```
/dev/sda2   40G   7.8G   31G   21%   /
```

```
sda      40.9G disk
+-sda1      1M part
+-sda2   40.9G part /
```

```
16K /home/findredhundred/os-lab1
```

`df -h` showed mounted filesystems and their free space. `lsblk` showed block devices such as disks and partitions. `du -sh` showed how much disk space my lab directory used.

```
/dev/sda2 on / type ext4 (rw,relatime)
devtmpfs on /dev type devtmpfs (...)
sysfs on /sys type sysfs (...)
```

`mount` showed which filesystem was attached to which point in the directory tree. In my case `/dev/sda2` was mounted at `/` and used the `ext4` filesystem.

`/dev` contained special device entries rather than normal text files. These entries give programs a file-like way to interact with devices and kernel drivers, which helped me understand the Unix idea that "everything is a file".

5.1 Observations

The root filesystem `/` was mounted on `/dev/sda2`. One useful device example is the real-time clock device, such as `/dev/rtc` or `/dev/rtc0`, which gives access to the system hardware clock. The idea of "everything is a file" means that many devices and system resources are exposed through file-like interfaces, so programs can interact with them using similar operations to normal files. This part showed the difference between filesystems, disks, partitions, mount points and device interfaces.

6 Wrap-up

This lab showed me the four main resources managed by an operating system: files, processes, memory and devices/I/O. I used `ls` to inspect files, `ps` to inspect processes, `free` to inspect memory and `df`/`lsblk` to inspect storage and devices. Overall, it made the things happening in the background of Linux much more visible and easier to connect to the OS concepts from class.

7 Conclusion

This laboratory work demonstrated how Linux acts as a resource manager by exposing and controlling files, processes, memory, and devices. Working directly with permissions, process information in /proc, memory statistics, filesystems, and device interfaces connected the theoretical role of an operating system to observable system behavior. Overall, the exercises made the main OS-managed resources easier to identify and showed how standard Linux tools can be used to inspect and manage them.